Modeling of Business Processes

Practicum Manual 2010

Written by Freek van den Berg (freek@vdberg.us)
for the Radboud University Nijmegen, The Netherlands

INTRODUCTION

This manual has been written for the course **Modeling of Business processes** and deals with covering the practical part of the course. During the practical part of the course, the Process Factory tool developed by Cordys will be extensively used. Cordys is an ERP company headquartered in Putten, The Netherlands which specializes in offering solutions in the so called cloud computing area. Due to their successful implementation and techniques, Google has decided to use Cordys' technology in the Google Apps product.

Every student attending the course needs to have access to an account (login name and password) and an application to work in (administrator rights). The former is provided in a welcome email from Cordys (which occasionally might end up in the spam folder). The just mentioned credentials can then be entered at http://tpfdemos.com/olc after which all the work (both the building and executing of the application) can be done online.

The process factory is a tool that is completely browser based (currently working best in Internet Explorer), which means no additional software needs to be installed on the client. A fancy property of the tool is that once something is defined, it becomes directly executable. Or in other words: requirements=software. Also the development and execution area are the same space. Although depending on the different rights that are granted to users, they may only experience either development or execution rights and corresponding GUI elements to utilize those. For the course however, every user gets full access to an application, giving the possibility to both development and execute (including testing) applications to its full extend. Note however, since this technology at this time of writing is very new and revolutionary, that the CPF software might not always be completely bug free and might respond occasionally in unexpected ways. In many cases mistakes are caused by the browser settings not being optimal (eg a too high privacy level might hinder the execution of web services) or information stored in the browser interfering with proper execution (eg cookies or cache information that is not applicable anymore). In the former cases lowering your privacy level might help and in the latter case restarting your browser and/or deleting cooking and cache might cause problems to go away. Even the browser version can have impact to how the CPF is run.

PRODUCT DESCRIPTION

The application that needs to be made (or in Cordys terms composed, due to Jan Baan's passion for classical music), encompasses a system that brings drivers and riders together. This system is called **RiderDriver** from now on. For more information the Giphouse (an internal university software engineering company virtually located at www.giphouse.nl) project named Epool held fall 2009 provides a lot of pointers. The assumption here is that drivers are driving cars all around the country. At random times riders need a transport from their current location to a given location, which in the old fashioned way is communicated by standing next to the road and holding up a sign with your final destination. In the modern way (read: RiderDriver system) this is communicated digitally (eg through SMS) to the system by the rider after which some automatic mechanism has to be put into practice. This mechanism (a business process) causes the lifter and an appropriate driver to be brought in touch.

Some properties that are valuable to make the application do its job are:

Driver's location – It's important to know where drivers are all the time in order to see whether they can mean anything for a new rider that needs to be taken somewhere (Google latitude is used here)

Interface to driver and rider – The system needs to communicate with both the driver and lifter and have them get them in touch (For the former SMS messages are used, while the latter is solved by using a two line phone connection, both provided by Voipbuster.com).

Fuel price – In order to have a decent payment model, the real time fuel price needs to be taken into account (A wrapped web service taking this information from the anwb.nl website provides a solution here)

Route planning – To determine whether a driver needs to make a big detour or not (in which the latter one is preferred of course) to serve the driver, route planning is required. The driver matching the lifter's route best can then be selected (Webservices.nl provides a trial account to achieve this for free)

Logging and emailing – Whenever communication needs to be done for debugging purposes or other user interactions as above, the output of a business process should be connected to a logging unit or email client. (These are also wrapped in a web service taking the message as input).

Postcode to street and city – In The Netherlands is normally convenient to store and ask for postcodes to identify a location. This is because every postcode/house number combination defines a unique address in The Netherlands. The postcodes are however not human friendly and when a driver or rider needs to be informed about a location, a street and city name are highly preferred. The conversion between the postcodes, street names and cities can be done using a remote postcode database. (Webservices.nl provides a trial account to achieve this for free)

Note that every property just mentioned ends with the implementation details between brackets. Doing this is very much unlike cloud computing in which everything is offered as a web service at any place on the world (through the internet).

A **web service** can be seen as a function taking input and generating output after a certain (often short) time. Accessing the web service leads to an action being performed, but generally the implementation is not important at all, as long as the result is good and fast enough (Eg. When you do a search on Google, many computers in the world might be accessed to process the query, but the average John Doe using Google does not care at all and just wants to have a certain result to his query within a certain time).

At the current practicum the implementation details are hidden by everything just mentioned being a distinct web service in a single directory of services. Just by adding one URL (with a WSDL definition) in the Process Factory, all the functionality can be used at once. In the later list of web services chapter in this document, this URL is provided and a semi formal definition of the services is given. Note that due to time constraints (only 1 ECTS for the practical course) and not wanting to deal with small implementation details, only fragments of the application will be built.

EXAMINATION

With respect to the course Modeling of Business Processes a three points scale grade will be used for the practical part, having respectively the following consequences:

Unsatisfactory: You will not be allowed to attend the final examination for the theory part of the course and therefore automatically fail the whole course. A second attempt for the practical part is scheduled just before the re-exam, which you may attend when you practical score has improved to satisfactory or good.

Satisfactory: You have passed the practical part of the course and the final grade of the course will solely be determined by the grade of the written exam

Good: Same as satisfactory, but 1.0 points bonus on your written exam score (eg 5.3 on theory exam results in a 6.3 final grade for the course)

In general the practical exam will not be impossible to pass, but some effort has to be taken. You are allowed to do the practical part alone, but working with groups of two people is highly recommended. Examination will take place at July 2nd, 2010 and will encompass a 15 minutes oral exam in which you demonstrate your product. The time slot of your exam can be obtained by signing in at the wanted time slot at Blackboard in the group page. This will also decide the composition of the groups.

As preparation to your oral exam you are supposed make the assignments as listed in the next part of the manual. In order to do so, you have received credentials to enter the Cordys Process Factory, in which you (and your group member) have administrator rights to an application. All the assignments can be performed in this application and no extra materials and/or products (such as a report) are expected from you.

ASSIGNMENTS

At the website (at http://www.erikproper.eu/courses/modelleren-van-bedrijfsprocessen) it shows that the practicum part consists of 7 meetings. The first two meetings provide lectures in which background information (about cloud computing and web services) and a demo of the CPF tool is given. The four meetings after that deal with finishing one assignment per meeting. There are no hard deadlines per assignment, but it is recommendable to finish an assignment before attending the meeting about the next assignment. At the final oral exam however (meeting number 7), you are supposed to have finished all the assignments. You are always allowed to make additions to what is mentioned in the assignments (in order to make it more likely to score a good mark or just for fun), but be sure to at least include the mandatory part.

Assignment 1 (Creating tables and importing data):

Create a table (Application object in CPF terminology) for a rider and another table for the driver. Try to think of all the attributes (such a name, mobile number, Google ID) that might turn out to be relevant for the RiderDriver application and at these to the corresponding tables (the next two assignments might provide you some valuable hints about what attributes are needed). After this you should be able to export the tables in CSV format and open them in a program on your computer (such as MS Excel) to edit them. Add at least 10 records for each table with fictional information. In order to help you a bit, some data (such as Google IDs and postcodes) has been provider in a chapter called "Fictional data" later in the manual.

Assignment 2 (Creating rider form):

Now that an application object has been created for the rider, it becomes possible to make a form and link it to the object. Create a detail form for the rider. Add four buttons to the form, respectively "send email", "send SMS", "get street" and "get city". Add two labels boxes to the form, named "message" and "result" The buttons should have the following effect on a click event:

Send SMS: Whatever is in the message field is send to the currently active rider. The result of this send action (normally OK) is placed in the "result" label box as a result.

Send Email: Same as Send SMS, except for the message being emailed instead of send through SMS.

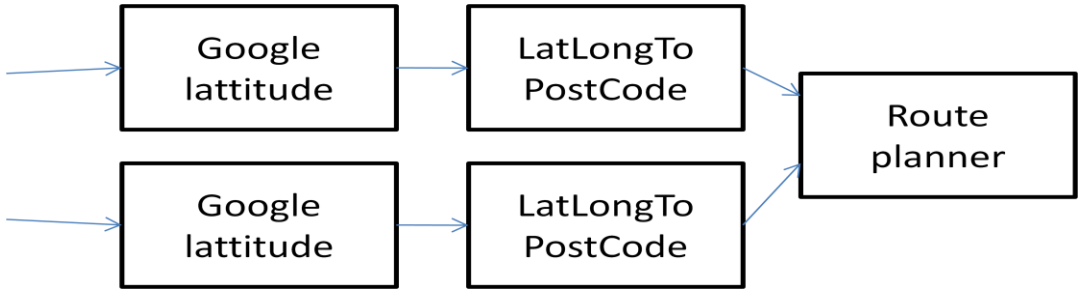
Get Street: The street name of the currently active rider should be updated, depending on the value of the postcode and house number.

Get City: Same as Get Street, except for the city field being updated instead of the street one.

Implementing the different buttons require the web services to be imported in the CPF environment first. The URL for this can be found in a later chapter named "List of web services". Once imported, they will remain available in the CPF environment forever and can thus be reused for assignment 3 and 4 as well.

Assignment 3 (Creating driver form):
Create a form for the driver application object in a similar way as the previous assignment. The buttons that need to be created on this form are “get location”, “show on map”, “plan route”, “connect through phone”. A widget location viewer needs to be added to the form to show a location and in addition to this label boxes named “destination post code”, “route description”, “another phone number” and “current location” are needed. The clicking on the buttons should have the following results respectively:
Get location: Using the Google ID the location should be derived and put in the label box current location
Show on map: The information that has been obtained with get location, is reused to be transferred to the Location viewer. Pressing the show on map button thus results into a location being plotted on the widget.
Plan route: The postcode from the active driver and the one from the corresponding label box (destination post code) are taken and used to calculate a route. The route description needs to be dumped into the route description label box.
Connect through phone: The active driver’s phone number and the phone number from the label box are taken and are used to both connect both people through a phone connection. This results into the driver’s phone ringing and the other person’s phone ringing and them hearing each other after they have both picked up the phone.

Assignment 4 (Using new web services):
Look for new web services on the internet (for example http://www.webservicex.net/ which has been found by Googling for ‘free web services’) and combine at least **three** of them to make a useful application for the RiderDriver system (hint: it might be useful to use http://www.soapclient.com to test whether the web services work and to get an impression of what results can be expected from them). Connect the web services through building a business process, which eventually leads to the creation of a new web service. Make a form with the input fields and the output field to test the newly created web service. An example of a composed web service is:



This web service takes two Google IDs (of two people being anywhere on earth) and returns how person A can reach person B, by providing a route description.

LIST OF WEBSERVICES

For this course some web services have made available for free (as long as not used extensively). Note that in reality one has to get a periodical membership or pay per use basis (eg sending an SMS normally costs 5-20 cents per message). The URL that is used for this course providing the WSDL file (the directory of services offered) is:
http://freek.vdberg.us/webservice/soap-server.php?wsdl
and testing the services quickly can be done at http://www.soapclient.com/soatptest.html

After importing this URL in the CPF the following services appear with corresponding semi formal definitions and parameters (in) and return value (out):

SendSMS: Sends an SMS containing a custom message to a cell phone anywhere in the world
In: Message: The SMS message to be sent up to 160 characters (eg Hello world)
In: Number: The number of the phone the message needs to be delivered to (eg +31xxx)
Out: The result of the delivery of the message (eg Ok)
(note that when this service is not activated, due to cost reasons, one can still obtain the result at http://freek.vdberg.us/webservice/sms.log)

SendMail: Sends an email to a valid email address with a message and subject
In: Message: The message to be emailed
In: Subject: The subject of the email
In: Email: A valid email address to which the message will be delivered (eg J@Exa.com)
Out: The result of the delivery of the message (eg Ok)

FuelPriceGasoline: The fuel price of gasoline in the Netherlands at current time
Out: The fuel price in Eurocents (eg 143.6)
(note that when this service is not activated, the return value will be "fuelpriceissupposedtobehere")

ConnectTwoPhones: Connects two phones which both have an international phone number
In: Phonenumber1: The number of the first phone that needs to be dialed (eg 00316…)
In: Phonenumber2: The number of the second phone that needs to be dialed (eg 00…)
Out: The result of the establishment of the connection (eg Ok)
(note that when this service is not activated, due to cost reasons, one can still obtain the result at http://freek.vdberg.us/webservice/connect.log)

SendToLog: Writes a message to the URL http://freek.vdberg.us/webservice/mvb.log which might come in handy for debugging purposes

In: Message: The message to be published on the URL

Out: The result of the publish action (eg Ok)

PostcodeToCity: Converts a Dutch postcode and house number to a city name

In: Postcode: A dutch postcode (eg 5432 AM)

In: Number: A dutch street house number beloging to the just mentioned postcode

Out: The city name (eg Cuijk)

(note that when this service is not activated due to cost reasons, the return value will be "cityof9999XX")

PostcodeToStreet: Converts a Dutch postcode and house number to a street name

In: Postcode: A dutch postcode (eg 5432 AM)

In: Number: A dutch street house number beloging to the just mentioned postcode

Out: The street name (eg Sleedoorn)

(note that when this service is not activated due to cost reasons, the return value will be "streetof9999XX")

Routeplanner: Gives a route description between two Dutch postcodes

In: Postcode1: The postcode of the departing location

In: Postcode2: The postcode of the goal location

Out: A route description between the departing location and destination

(note that when this service is not activated due to cost reasons, the return value will be "routefrom9999XXto9999XX")

GoogleLatitudeSpaceLongitude: Converts a Google ID to a latitude and longitude

In: Google ID: The Google ID of the person whose location needs to be tracked

Out: A location in the following format: [latitude]space[longitude]

GoogleLatitude: Converts a Google ID to a latitude

In: Google ID: The Google ID of the person whose location needs to be tracked

Out: A latitude of the location

GoogleLongitude: Converts a Google ID to a longitude

In: Google ID: The Google ID of the person whose location needs to be tracked

Out: A longitude of the location

FICTIONAL DATA

Google latitude IDs:

- 7876103850984392975, 4364000254596359846, - 5575903466058229269,
- 5743069642080515 96, 4531022096563578555, 8129199253614500423,
1227945836344465043, 3438371737135740828, - 8816352646096526591

More can be obtained through smartly investigating the following URL:
http://www.google.nl/search?hl=nl&rls=com.microsoft:en-us:IE-SearchBox&q=site:http://www.google.com/latitude/apps/badge/api%3Fuser&start=270&sa=N

Last names:

AARSE, ABRAHAM, ABRESCH, ACHTERSTRAAT, ADELAER, ADENROCK, ADLER, ADRIAEN, ADRIAENS, ADRIAENSZ.VERMEULEN, AERTS, AERTS., AL, ALLAN, ALLEN, AMONS, AMPTMAN, ANOKKEE, ANTHONISZ., ARIENS, ARISDR., ASSELAER, ASSELMAN, AUDEREGG, AUGUSTINUS

Source:
http://www.geneaworld.com/bosschaert/surnames.html

First names:

Aa Aad Aadil Aadyish Aage Aai Aaielt Aaike Aaildert Aailert Aailt Aaissa Aaite Aal Aalbert Aalderik Aalderk Aaldert Aaldrikus Aalf Aalje Aalk Aalke Aalmoed Aalrik Aalse Aalt Aaltienus Aaltjo Aalwijn Aalzen Aam Aar Aard Aardewijn Aardsche Aardse Aarjen Aarke Aarn Aarnoud Aarnout Aaron Aart Aas Aat Aauwke Ab Abas Abbas Abbe Abbo Abdallah Abdel Abdelah Abdelrahim Abdessamad Abdias Abdoellah Abdon Abdoul Abdoulah Abdoullah Abdul Abdulah Abdullah Abe Abeke Abel Abele Abelkin Abelone Abeltje Aberico Abertus Abia Abiegel Abigael Abigal Abiner Abner Abo Abraham Abrecht

Source:
http://www.voornamen.com/jongens.php

Postcode/house number combinations:

1091AS 2/4, 1078VV 56/HS, 1024NZ 405, 1082KR 3/123, 1053ES 271

Source:
http://www.detelefoongids.nl/zoeken/plaats-amsterdam/w-jansen